

Deep Web Research — Solutions for Lava's Standing Challenges (2026-07-02)

Author: research subagent. Standing operator directive: "Perform frequent deep web researches about solutions for all of our challenges, new ideas, game-changing approaches, and various opensource codebases and services we can incorporate into our System!"

Anti-bluff note. Every project/version/date below was pulled from a URL that was actually fetched during this session (listed in each section's *Sources*). Where a fact could not be confirmed from a primary source it is prefixed UNVERIFIED:. Version numbers and dates reflect what the fetched page reported on 2026-07-02 and should be re-checked before adoption. No claim here is a substitute for the §6.J/§6.L requirement that any incorporated tool be proven on a real device / real service before it gates anything.

Topic 1 — Live HTML fixture refresh + anti-Cloudflare

Lava's problem: keeping scraper test fixtures (and the live :proxy / lava-api-go scrapers) working against sites behind Cloudflare — notably rutor.info, which currently returns 403/501 to plain HTTP clients. The 2026 landscape has shifted: the classic FlareSolvrr sidcar still works for *light* JS challenges but its CAPTCHA/Turnstile solvers are non-functional, and TLS-fingerprint impersonation has become the cheaper first line of defense.

Findings

- **FlareSolvrr** (github.com/FlareSolvrr/FlareSolvrr, **MIT**). Latest release reported as **v3.5.0 (2026-05-26)**; 14.5k stars, still receiving releases. A Dockerized proxy that returns cookies+UA after solving a challenge in a real browser. **Critical limitation confirmed on the README:** *"At this time none of the captcha solvers work"* — it returns Captcha detected but no automatic solver is configured on Turnstile / 2026 Managed Challenges. Still viable as a sidecar for sites that only throw a light JS challenge (this is a good fit for a container in Lava's stack because it speaks a simple JSON HTTP API and is language-agnostic). *How Lava could use it:* run as a submodules/containers-managed sidecar; the Go/Kotlin scraper POSTs `{cmd:"request.get", url:...}` to `:8191/v1` and uses the returned cookies for the real fetch. Only helps where the tracker uses a light JS challenge, not Turnstile.
- **Byparr** (github.com/ThePhaseless/Byparr, **GPL-3.0**). Latest release **v2.1.0 (2026-02-08)**. A **drop-in FlareSolvrr replacement**: same `:8191/v1` JSON API, shipped as `ghcr.io/thephaseless/byparr:latest`. Secondary sources say it is Camoufox-backed (hardened Firefox) with a higher success rate than FlareSolvrr on Turnstile; the repo itself only promises *"greatly increases the chance"* (honest, no guarantee). **License caveat:** GPL-3.0 — fine as an unmodified sidecar container, but do NOT vendor its source into Lava (which is not GPL). *How Lava could use it:* identical wiring to FlareSolvrr but with a better Turnstile hit-rate; swap the sidecar image, keep the client code. Best current open-source option for a browser-based challenge solver.
- **curl_cffi** (github.com/lexiforest/curl_cffi, **MIT**). Latest **v0.15.1b2 (2026-06-05)**, actively maintained, 37 preset browser fingerprints (Chrome/Safari/Firefox families), and — since v0.15.1 — fingerprints can be updated without upgrading the library. It is the Python binding; the **standalone github.com/lexiforest/curl-impersonate** distribution is a patched curl that impersonates browser TLS/JA3/HTTP2 (and HTTP/3) fingerprints. This is the *cheap* first line: many Cloudflare 403s are pure TLS-fingerprint blocks that never reach a JS challenge, and a fingerprint-correct request just succeeds with no browser. *How Lava could use it:* the `lava-api-go` scraper is Go, not Python — the actionable path is the **curl-impersonate binary/shared-lib as a fetch backend** (invoke the impersonate curl or link the boringssl-patched build) OR a thin Python `curl_cffi` sidecar for the fixture-refresh script. For the JVM app side, the analogous idea is documented under "okhttp custom

fingerprints" in the `curl_cffi` docs (they show `okhttp JA3`), i.e. Lava's `OkHttp` client can be tuned toward a browser-like fingerprint. Recommend `curl-impersonate` as the **fixture-refresh fetcher** first, before reaching for a browser.

- **Scrapling** (Python framework; secondary-source only this session — see caveat). Reported **v0.4.8 (2026-05)**: one library with three fetchers (TLS-impersonated HTTP → Camoufox with an auto-Turnstile solver → full Playwright), letting a fixture-refresh script escalate only as far as a given tracker needs. UNVERIFIED (repo not fetched): exact license and repo URL — verify before use. *How Lava could use it*: the `scripts/check-fixture-freshness.sh` companion refresher could call a small Scrapling script that auto-escalates per tracker.
- **cloudscraper** (`github.com/VeNoMouS/cloudscraper`, Python). **Mixed/declining signal**. Some sources call the mainline fork stale; a `cloudscraper25` fork is cited as the currently-maintained one. Open-source solvers of this class lag Cloudflare changes. *Recommendation*: **do not adopt** as a primary — it is the weakest of the options here for 2026 Cloudflare.
- **undetected-chromedriver** / **nodriver** and **Playwright stealth** — the in-process browser route (no separate container). Higher operational weight; only needed when a tracker requires full browser interaction. For Lava, a shared **Byparr sidecar** is a cleaner fit than embedding a browser in the Go/Kotlin process.

Recommendation for Lava

1. **Two-tier fetch strategy for the scrapers and the fixture-refresh script**. Tier 1: **curl-impersonate** / **curl_cffi** with a current Chrome fingerprint — cheapest, no browser, likely clears `rutor.info`'s TLS-fingerprint 403 outright. Tier 2 (only if Tier 1 gets a JS/Turnstile challenge): a **Byparr** sidecar container (FlareSolverr-API-compatible, better Turnstile hit-rate than FlareSolverr, GPL-3.0 used *unmodified as a container* so no license contamination), managed by `submodules/containers` per §6.V/§6.X.
2. Keep the tool **out of the app**: this belongs in the backend scraper + the fixture-refresh tooling, never shipped in the Android APK.
3. Anti-bluff: wire a contract test (per §6.A) asserting the fixture-refresh fetcher actually returns a 200 + expected tracker markup for each tracker, and that a deliberately-wrong fingerprint

reproduces the 403 — so the refresher can't silently "succeed" on a Cloudflare error page (the classic stale-fixture bluff).

Sources

- <https://github.com/FlareSolverr/FlareSolverr>
 - <https://github.com/ThePhaseless/Byparr>
 - https://github.com/lexiforest/curl_cffi
 - <https://curl-cffi.readthedocs.io/en/latest/impersonate/customize.html> (okhttp custom fingerprints)
 - <https://github.com/VeNoMouS/cloudscraper>
 - <https://godberrystudios.com/posts/byparr-scrapping-flaresolverr-cloudflare-bypass-2026/> (secondary: Byparr/Scrapling landscape)
 - <https://www.zenrows.com/blog/byparr> (secondary: Byparr = Camoufox-backed)
-

Topic 2 — Opensource tracker parsers / metadata libraries

Findings

- **TorAPI** (github.com/Lifailon/TorAPI, MIT). **This is the standout find.** An unofficial **backend REST API + RSS** for exactly Lava's target set: **RuTracker, Kinozal, RuTor, and NoNameClub (NNMClub)**. Node.js/Express, latest **v0.5.2 (2024-10-09)**. Self-hostable via `lifailon/torapi:latest` Docker image / Compose. Search by title or ID, category filtering, magnet links, .torrent download, plus enriched metadata (IMDb/Kinopoisk links, posters). **No tracker auth or token required for search**; proxy address/port/user/pass are configurable via env vars. *How Lava could use it:* the four trackers TorAPI covers overlap Lava's provider matrix almost exactly. It is a **reference implementation of the parsers Lava maintains by hand** — even without adopting it as a runtime dependency, its parsing logic (selectors, category maps, RSS shapes) is a cross-check to keep Lava's `core/tracker/*` parsers honest and a fast way to see how a 4th/5th tracker is scraped. Adoption caveat: it is Node.js (a new runtime in Lava's stack) and last tagged 2024-10 — treat as a **parser-reference + optional sidecar**, not a frozen dependency, and confirm it still tracks current tracker HTML before trusting it.

- **Prowlarr / Jackett Cardigann YAML indexer definitions**
(github.com/Prowlarr/Indexers, and github.com/Jackett/Jackett). Community-maintained declarative **YAML (Cardigann) scraper definitions** synced Jackett→Prowlarr daily via GitHub Actions. Cardigann supports HTML/JSON/XML parsing. A Russian-tracker example set lives at github.com/msergein/ru-cardigann-yml (RuTracker in Prowlarr Cardigann YAML, with release-name reformatting for Radarr/Sonarr). *How Lava could use it*: the Cardigann YAML for a given tracker is a **field-level spec of that tracker's search rows and categories** — a maintenance aid and a potential model for making Lava's own tracker definitions data-driven rather than hand-coded Kotlin (aligns with §6.R no-hardcoding: move selectors/category maps into config). License varies per repo — Jackett is GPL-2.0, Prowlarr GPL-3.0; use the *YAML definitions as reference/data*, not the GPL engine code, to avoid license contamination.
- **qBittorrent search-engine plugins** (community; Python). A widely-used plugin set covering **rutracker**, **rutor**, **nnmclub**, **kinozal** (secondary source reports ~133 stars, updated 2025-07). UNVERIFIED (repo not fetched this session): exact canonical repo URL + license — verify before relying on it. Same value proposition as TorAPI/Cardigann: parser reference for the same four trackers.
- **Torrent metadata parsing libraries.**
 - **Go**: github.com/anacrolix/torrent (full BitTorrent client) ships reusable sub-packages — **bencode** (Marshal/Unmarshal, json-like API) and **metainfo** (ParseMagnetUri, .torrent parse). This is the mature Go choice and fits lava-api-go directly. License: check repo (historically MPL-2.0 — UNVERIFIED this session, confirm).
 - **.NET**: github.com/Krusen/BencodeNET (bencode + .torrent read/write + magnet generation) — not Lava's stack but a clean reference.
 - **Kotlin/JVM**: no dedicated, actively-maintained Kotlin bencode/torrent library surfaced in this session's searches. UNVERIFIED: a first-class Kotlin lib may exist — not found. For the JVM app, the pragmatic path is a small hand-rolled bencode reader (the format is tiny) or a Java bencode lib, since Lava mostly needs magnet/.torrent field extraction, not a full client.

Recommendation for Lava

1. **Adopt TorAPI as a parser-reference (and optional self-hosted sidecar).** It targets Lava's exact four trackers, is MIT, and needs no

credentials for search — it is the single best external cross-check that Lava's RuTracker/RuTor/Kinozal/NNMClub parsers still match live HTML. Wire a periodic diff (Lava parser output vs. TorAPI output for the same query) into the fixture-freshness tooling to catch silent tracker-HTML drift.

2. **Mine Prowlarr/Jackett Cardigann YAML** for the same trackers to (a) validate Lava's category maps and selectors and (b) prototype a **data-driven tracker-definition format** (§6.R-aligned: selectors in config, not hardcoded).
3. **Use anacrolix/torrent's bencode+metainfo in lava-api-go** for any magnet/.torrent handling instead of hand-rolling — confirm the license first.

Sources

- <https://github.com/Lifailon/TorAPI>
 - <https://github.com/Lifailon/TorAPI/blob/main/docs.md>
 - <https://github.com/Prowlarr/indexers>
 - <https://github.com/msergein/ru-cardigann-yml>
 - <https://wiki.servarr.com/prowlarr/cardigann-yml-definition>
 - <https://pkg.go.dev/github.com/anacrolix/torrent/bencode>
 - <https://pkg.go.dev/github.com/anacrolix/torrent/metainfo>
 - <https://github.com/anacrolix/torrent>
 - <https://github.com/Krusen/BencodeNET>
-

Topic 3 — Containerized Android emulator farms (for the §6.X CI gate)

Lava's §6.X/§6.V/§6.AH mandate: emulators must boot **inside a container/VM managed by the Containers submodule**, never host-direct. The standing blocker (§6.X-debt, §6.AH-debt) is that a Linux container needs /dev/kvm, which the darwin/arm64 podman VM does not expose. This topic surveys what the field ships so the Containers submodule can converge on a proven design.

Findings

- **Google android-emulator-container-scripts** (github.com/google/android-emulator-container-scripts, **Apache-2.0**). The **official**, most-authoritative reference. Publishes prebuilt emulator images at us-docker.pkg.dev/android-emulator-268719/images/ spanning

API 30–37 (google_apis, google_apis_playstore, and 16KB-page variants). Exposes **adb over forwarded port 5555**, a **gRPC** control service, and a **WebRTC** video bridge + Envoy TLS proxy for browser access. **Hard constraints (confirmed on the repo):** *"The images will not run in docker on mac or windows"*, **KVM is mandatory** (bare metal, nested virtualization, or a cloud instance with nested virt), and the tooling is **still experimental** (Python ≥ 3.10 , run in a venv). *How Lava could use it:* this is the **canonical design the Containers submodule's Containerized emulator path should mirror** — image layout, adb-5555 forwarding, and especially the **gRPC + WebRTC remote surface** (which would give Lava headless per-AVD control + video capture for §11.4.128 recording). Its Linux+KVM-only reality also **confirms Lava's §6.AH-debt is a real, industry-wide constraint, not an agent excuse**: the accepted answer is a **Linux x86_64 KVM gate host**, not host-direct on macOS.

- **budtmo/docker-android** (github.com/budtmo/docker-android). The most popular community image: Android emulator in Docker with **noVNC** (web view), **video recording**, and Appium integration. Prebuilt images for **Android 9.0–14.0** (e.g. budtmo/docker-android:emulator_13.0). Requires `--device /dev/kvm` on Linux (kernel ≥ 4.10). UNVERIFIED this session: exact license (historically MIT/Apache — confirm on repo). *How Lava could use it:* fastest way to get a **known-good containerized-emulator + built-in screen recording + VNC** reference to compare against the Containers submodule's implementation; its video-recording feature is directly relevant to Lava's §11.4.128 always-on device-recording debt.
- **Genymotion** (SaaS + **Self-Hosted**; genymotion.com). **Commercial/proprietary**, not open-source. Runs Android VMs in the cloud or in your private cloud/on-prem. Already familiar to Lava (the CLAUDE.md history references a Genymotion VM as a gate host). *How Lava could use it:* a **paid fallback gate host** when a Linux+KVM box isn't available — but it does **not** satisfy the "open-source, self-hostable, Containers-submodule-driven" preference, so treat as escape hatch, not strategy.
- **OpenSTF / DeviceFarmer** (github.com/DeviceFarmer/stf). **OpenSTF is effectively dead** (last release v3.4.1, Android ≤ 9 , project abandoned ~2020); **DeviceFarmer** is the volunteer-maintained successor. **Important scope point:** STF/DeviceFarmer is **remote device access (view/tap/type/ install)**, **NOT a test runner** — it does not execute Espresso/Appium/Maestro. *How Lava*

could use it: not a fit for the automated §6.X gate (no test execution); only relevant if Lava ever wants a human-remote-control device lab. **Recommend against** for the CI gate.

Recommendation for Lava

1. **Model the Containers submodule's Containerized emulator path on Google's android-emulator-container-scripts** (Apache-2.0, official): its **image layout** + **adb-5555** + **gRPC control** + **WebRTC video** are exactly the primitives §6.X and §11.4.128 need, and its Linux+KVM-only requirement is authoritative confirmation that the darwin/arm64 gap is structural.
2. **Provision a Linux x86_64 host with /dev/kvm** as the constitutional gate host — this is the accepted industry answer and directly closes §6.AH-debt (no host-direct fallback needed once a real KVM box exists). Every fetched primary source agrees KVM-on-Linux is the path; macOS/Windows containers are unsupported for hardware-accelerated emulation.
3. **Borrow budtmo/docker-android's built-in video recording** pattern to satisfy the §11.4.128 always-on device-recorder debt, whether or not its image is adopted wholesale.
4. **Do not use OpenSTF/DeviceFarmer for the gate** — it's device-access, not a test runner.

Sources

- <https://github.com/google/android-emulator-container-scripts>
 - <https://github.com/google/android-emulator-container-scripts/blob/master/REGISTRY.MD>
 - <https://github.com/budtmo/docker-android>
 - <https://www.genymotion.com/product-self-hosted/>
 - <https://devicelab.dev/blog/openstf-devicefarmer-alternative>
 - <https://dev.to/omnarayan/openstf-is-dead-the-best-alternative-for-mobile-device-labs-in-2025-2j5o>
-

Topic 4 — Autonomous mobile UI QA + vision analysis

Lava already drives Compose UI Challenge Tests; the operator's video-issue workflow (§6.AK: every manual-testing-video issue must gain a reproduce-first device Challenge) makes **AI-driven UI exploration**

and screen/video analysis directly useful for turning operator videos into repro tests and for catching visual defects the deterministic Challenges miss.

Findings

- **Maestro** (github.com/mobile-dev-inc/maestro, **Apache-2.0**). Latest CLI **2.6.1 (2026-06-12)**, very active. **YAML flows** (`launchApp`, `tapOn`, `assertVisible`), self-hosted CLI (`curl -fsSL https://get.maestro.mobile.dev | bash`), drives **Android emulators via adb with no app-code changes**, built-in flakiness tolerance/smart-waiting. **Ships experimental LLM/vision commands directly relevant to Lava:**
 - **assertNoDefectsWithAI** — screenshots the screen and asks an LLM to flag common visual defects (text/UI cut off, overlapping, off-center). This is a **smoke test for exactly the class of §6.AB/§6.Z bugs Lava keeps shipping** (the white-placeholder logo, the cut-off subtitle).
 - **assertWithAI** — natural-language visual assertion via LLM screenshot upload (good for the "is the brand mark actually rendered in Lava red, not white?" check that pixel assertions kept missing).
 - **extractTextWithAI** — pulls dynamic/imagery text (e.g. CAPTCHAs, banners) into `aiOutput`.
 - Plus **AI test analysis** reports (HTML+JSON) for a whole run. There is also a **KMaestro Kotlin DSL** (github.com/cnrture/KMaestro) for writing Maestro flows type-safely from Kotlin — a natural bridge for Lava's JVM stack. *How Lava could use it:* add a **Maestro flow layer alongside the Espresso Challenges** — YAML flows are far faster to author from an operator video than Compose test code, and `assertNoDefectsWithAI` / `assertWithAI` give a **vision-based acceptance gate** that catches non-crashing visual regressions the deterministic tests miss (the recurring §6.AB failure class). Self-hosted, Apache-2.0, adb-based — fits the Containers-driven emulator gate.
- **DroidRun** (repo github.com/droidrun/droidrun **redirects to github.com/droidrun/mobilerun**, MIT). Latest **v0.6.8 (2026-06-22)**, very active (~1,267 commits). An **LLM-agnostic mobile agent** (OpenAI, Anthropic, Gemini, Ollama, DeepSeek, OpenRouter) that controls a device via **adb + a "Portal" companion app** (accessibility service), reads the **accessibility tree** (fast, not vision-dependent) with optional **--vision screenshot mode** and **--**

reasoning manager-executor planning. Secondary sources report a **91.4% score on Google's AndroidWorld benchmark** and a 43% success rate across an independent 65-task suite (strongest of those tested). *How Lava could use it:* **exploratory / autonomous QA** — point it at the debug APK on the containerized emulator with natural-language goals ("onboard a no-auth provider and run a search") to surface flows that crash or dead-end, feeding \$6.AK reproduce-first Challenges. Note it needs an LLM API key and installs a companion accessibility app; keep it in the QA harness, never in the shipped app.

- **Fastbot2 / Fastbot_Android** (github.com/bytedance/Fastbot_Android). **Model-based GUI fuzzer** (ByteDance) enhanced with reinforcement learning; runs via **adb shell with no app instrumentation** (works on real Android 5–14 and emulators), ~12 actions/sec, emits crash/ANR logs (`/sdcard/crash-dump.log`, `/sdcard/oom-traces.log`) and activity-coverage metrics. Published research: deployed in ByteDance CI ~2 years, caught 50.8% of developer-fixed crash bugs on Douyin/ Toutiao; outperforms Monkey/Ape/Stoat on coverage + fault detection. UNVERIFIED this session: exact license + last-release date (the fetch showed 50 commits but no dated release — confirm). *How Lava could use it:* a **stability/crash-hunting monkey on steroids** for the \$6.Z cold-start / crash-survival class — run it in the containerized emulator against the debug APK to shake out onCreate/Hilt/Compose-init crashes before distribute. Complements (does not replace) the deterministic Challenges.
- **DroidBot** (github.com/honeynet/droidbot, academic). Lightweight **UI-guided input generator**, no instrumentation, builds an on-the-fly state-transition model. Older/academic; Fastbot2 is the more current model-based option. *How Lava could use it:* lower priority than Fastbot2 for the same crash-hunting job.
- **Appium 2 visual/OCR** (github.com/appium/appium, Apache-2.0). **images-plugin** (OpenCV pixel/region image comparison + `getImagesSimilarity`) and a **Tesseract-based appium-ocr-plugin** (OCR endpoint on the current screenshot). Appium has **built-in Android screen recording** for failed-test debugging. Note: Appium's image comparison is **pixel-based (OpenCV)**, prone to false positives on anti-aliasing/color shifts — it is *not* an LLM vision engine (Maestro's `assertWithAI` is closer to what Lava wants for "does this look right"). *How Lava could use it:* if Lava ever needs deterministic pixel/region matching or OCR of a rendered screen

(e.g. verifying the logo pixels are red), the images-plugin + ocr-plugin are the self-hosted, license-clean route; otherwise Maestro's AI commands are the lighter answer.

Recommendation for Lava

1. **Add Maestro (Apache-2.0) as a second UI-test layer** focused on **vision acceptance**: `assertNoDefectsWithAI` + `assertWithAI` on the key onboarding/search/provider screens is a direct, low-effort mechanical guard against the recurring §6.AB/§6.Z visual-regression class (white logo, cut-off text, gate-bypass). YAML flows are also the fastest way to convert an operator §6.AK video into a repro flow. Run it on the same Containers-driven emulator gate.
2. **Add Fastbot2 as a crash-hunting monkey** in the §6.Z pre-distribute gate — adb-driven, no instrumentation, emits crash/ANR logs; cheap insurance against cold-start crashes that the deterministic Challenges don't randomly hit. (Confirm its license first.)
3. **Evaluate DroidRun (MIT) for autonomous exploratory QA** — LLM-agnostic (can use the operator's existing Anthropic/Gemini keys, or local Ollama), reads the accessibility tree, optional vision. Use it in the QA harness to discover broken flows → feed §6.AK reproduce-first Challenges. Keep its companion accessibility app out of the shipped APK.
4. **Keep Appium images-plugin/ocr-plugin in reserve** for deterministic pixel/OCR checks where an LLM assertion is overkill or non-reproducible.
5. Anti-bluff guard for all four: an AI/vision assertion is only a gate if it is **falsifiable** (§6.J/§6.AB.3) — before trusting `assertNoDefectsWithAI` or a DroidRun run, prove it goes RED on a deliberately-broken build (monochrome logo, cut-off text). An LLM that rubber-stamps a broken screen is the exact §6.AK bluff, just at a new layer.

Sources

- <https://github.com/mobile-dev-inc/maestro>
- <https://docs.maestro.dev/api-reference/commands/assertnodefectswithai>
- <https://docs.maestro.dev/api-reference/commands/assertwithai>
- <https://docs.maestro.dev/api-reference/commands/extracttextwithai>
- <https://docs.maestro.dev/maestro-flows/workspace-management/ai-test-analysis>
- <https://github.com/cnrture/KMaestro>

- <https://github.com/droidrun/mobilerun>
 - https://github.com/bytedance/Fastbot_Android
 - <https://github.com/appium/appium/blob/master/packages/images-plugin/docs/image-comparison.md>
 - <https://github.com/jlipps/appium-ocr-plugin>
-

Cross-cutting takeaways

- **Cloudflare is now a two-tier problem:** TLS-fingerprint impersonation (curl-impersonate) solves the cheap cases; a browser sidecar (Byparr > FlareSolverr in 2026) solves JS/Turnstile. Adopt both tiers in the backend + fixture-refresh tooling, neither in the app.
- **TorAPI is the highest-leverage single find** — an MIT, self-hostable API covering Lava's exact four trackers, usable as a parser cross-check to end silent fixture drift.
- **The \$6.X/\$6.AH emulator gap is industry-confirmed structural:** Google's own container scripts are Linux+KVM-only and refuse to run on mac/windows. The clean fix is a Linux x86_64 KVM gate host; model the Containers submodule on Google's Apache-2.0 scripts (adb-5555 + gRPC + WebRTC video).
- **Maestro's assertNoDefectsWithAI is a near-exact mechanical guard** for the visual-regression class Lava keeps shipping (white logo / cut-off text) — Apache-2.0, self-hosted, adb-based, and it layers on top of the existing Challenges rather than replacing them.
- **Every recommendation stays subordinate to \$6.J/\$6.L:** any adopted tool must be proven falsifiable (goes RED on a real defect) on the real device gate before it is allowed to gate a distribute.